

임베디드 제어 및 로봇 SW 개발 포트폴리오

대표 기술
임베디드 제어 & 로봇 SW 개발

개인정보
성명 : 임청수
생년월일 :
연락처 :
E-mail : fclcdnd@naver.com

최종 학력 사항	
재학기간	학교명 및 전공
2009.03 ~ 2015.02	남부대학교/호텔조리학과

교육 이수 사항			
교육명	내용	교육기관	교육 일정
가전R&D를 위한 임베디드&지능로봇 개발자	회로시스템분석및설계, 펌웨어 개발, 객체지향프로그래밍, AI 영상 인식 구현, 임베디드 오픈 플랫폼 활용, 기구설계, ROS 활용	대한상공회의소 광주 인력 개발원	2025-12-16 ~ 2026-06-02

자격 사항		
자격증명	취득일	시행기관
운전면허 보통 대형	2008.12	전남 경찰청
양식조리기능사	2014.01	한국기술자격검정원
MOS Office Excel2010	2017.09	YBMIT

전문설계 소프트웨어 능력	
언어(툴)	활용능력
C언어	STM32 기반 임베디드 펌웨어 개발, UART-PWM-ADC-I2C 활용 센서 및 모터 제어
Python	OpenCV 영상처리, YOLOv8 객체 인식, Flask 기반 웹 모니터링 시스템 개발
STM32	STM32CubeIDE 기반 MCU 제어, Timer Interrupt 및 주변장치 연동
Raspberry Pi	Linux 기반 임베디드 시스템 구축, STM32 연동 및 영상처리 시스템 구현
ROS2	ROS2 Humble 기반 노드 통신, SLAM Toolbox, Nav2 자율주행 구현
micro-ROS	ESP32-S3와 ROS2 연동, UART 기반 Topic 송수신 구현
Linux	Ubuntu 및 Raspberry Pi OS 환경 구축, ROS2 및 Docker 개발환경 운영
OpenCV / YOLOv8	객체 탐지, HSV 기반 영상 분석, 실시간 비전 시스템 개발
EasyEDA	회로도 설계 및 PCB 설계 경험
SolidWorks	3D 모델링, 기구 설계 및 3D 프린터 출력 경험

프로젝트 수행사항			
번호	프로젝트명	프로젝트 내용	수행기관
1	초음파 센서	초음파 발진 및 수신부에서 받은 거리 값을 FND로 출력	대한상공회의소 광주 인력 개발원
2	Smart Grip Car	STM32 기반으로 초음파 센서와 조이스틱을 활용한 원격 제어 및 장애물 감지 기능을 구현한 지능형 차량 시스템	대한상공회의소 광주 인력 개발원
3	Vision AI 기반 스마트 푸드재고 관리 및 모니터링 시스템	Vision AI 기반 실시간 잔량 분석 및 데이터 로그 작성	대한상공회의소 광주 인력 개발원
4	Smart Grip Car Ver1.1	라즈베리 파이 4와 STM32를 연동한 YOLOv8 기반 실시간 객체 인식 및 웹 UI 원격 제어 자율 이송 로봇 시스템	대한상공회의소 광주 인력 개발원
5	ROS2 기반 코봇 자율주행 시스템 개발	ROS2 Humble, micro-ROS, LiDAR, Nav2를 활용하여 실내 지도 생성(SLAM) 및 목표 지점 자율주행 기능을 구현한 코봇(Co-Bot) 자율주행 시스템 개발	대한상공회의소 광주 인력 개발원

프로젝트 기술서

작성자
임청수

1	프로젝트명: 초음파 거리 측정기
수행 기간	2026. 01. 06 ~ 2026. 01. 07
담당 역할	Analog / Digital 하드웨어 분석 및 Bare PCB 기판 실장
수행 목표	초음파 발진 및 수신부에서 받은 거리 값을 FND로 출력
사용 기술	1. NE555 Timer를 이용한 발진회로 설계 2. Op-Amp 증폭기 설계 3. 다이오드를 이용한 평활회로 설계 4. 슈미트-트리거 Not gate 사용
세부 수행 내용	
1. 개요 가. 목적 - 초음파 센서를 이용한 거리 측정기 제작 나. 소개 - 초음파를 송신, 수신해 그 시간을 거리로 계산해서 표현 2. 프로젝트 블록도 가. 시스템 블록 다이어그램 <pre>graph LR; A[초음파 발진기] --> B[초음파 송신]; B --> C[초음파 수신]; C --> D[검파]; D --> E[신호 검출]; E --> F[시간 게이트 측정]; F --> G[측정 펄스 발진기]; G --> H[카운터 클리어 펄스 래치 클리어 펄스]; H --> I[시간 측정]; I --> J[FND 출력];</pre>	

세부 수행 내용

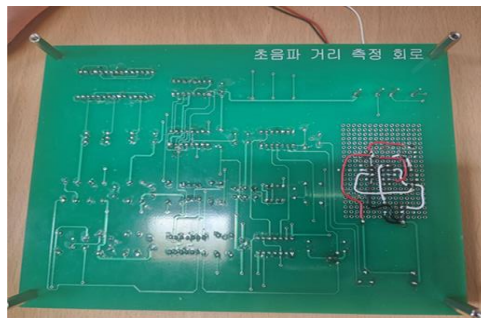
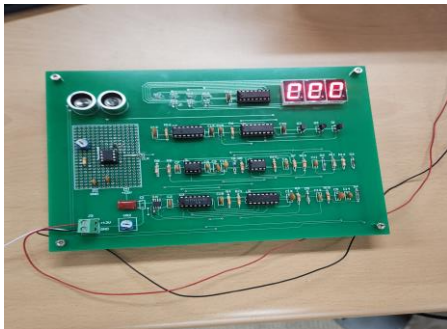
3. BOM (Bill Of Materials)

번호	재료명	규격(치수)	단위	소요량	비고
1	Chip IC	NE555(SMD TYPE)	개	1	
2	IC	NE555	개	1	
3	IC	4584	개	1	
4	IC	4011	개	1	
5	IC	RC4558	개	1	
6	IC	LM358	개	1	
7	IC	4069	개	1	
8	IC	4553	개	1	
9	IC	4511	개	1	
10	IC소켓	8PIN(DIP)	개	3	
11	IC소켓	14PIN(DIP)	개	3	
12	IC소켓	16PIN(DIP)	개	2	
13	저항	150[k Ω], 1/4[W], 1%	개	1	
14	저항	10[k Ω], 1/4[W], 1%	개	8	
15	저항	2[M Ω], 1/4[W], 1%	개	1	
16	저항	8.2[k Ω], 1/4[W], 1%	개	1	
17	저항	1[M Ω], 1/4[W], 1%	개	2	
18	저항	47[k Ω], 1/4[W], 1%	개	1	
19	저항	1[k Ω], 1/4[W], 1%	개	2	
21	CHIP 저항	SMD 330[Ω] (2012)	개	7	
22	마일러콘덴서	0.047[μ F]	개	1	
23	세라믹콘덴서	680[pF]	개	1	
24	세라믹콘덴서	100[pF]	개	1	
25	세라믹콘덴서	0.1[μ F]	개	9	

번호	재료명	규격(치수)	단위	소요량	비고
26	세라믹콘덴서	0.01[μ F]	개	3	
27	세라믹콘덴서	0.001[μ F]	개	4	
28	세라믹콘덴서	0.0047[μ F]	개	1	
29	반고정저항	47[k Ω]	개	1	
30	반고정저항	10[k Ω]	개	1	
31	TR	A1015	개	3	
32	FND	500	개	3	
33	초음파센서	SE-400ST160	개	1	
34	초음파센서	SE-400SR160	개	1	
35	Diode	1SS106	개	2	
36	Diode	1N4148	개	3	
37	컨넥터	녹색단자(2P)	개	1	
38	기판	Bear PCB	장	1	

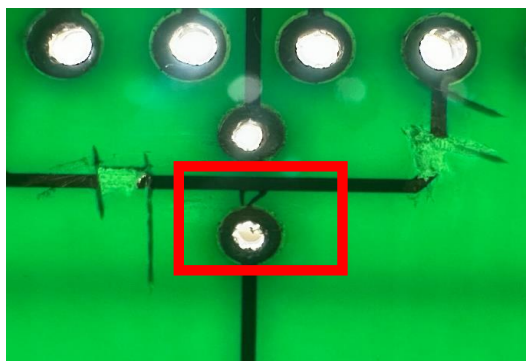
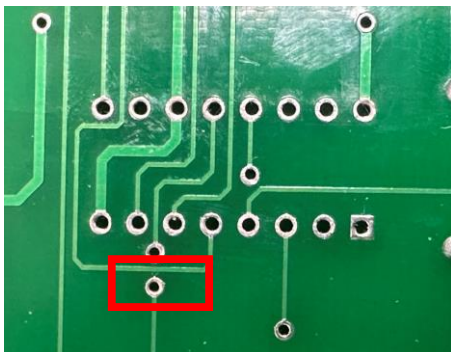
4. 제작 결과

가. 원인을 파악하지 못해 FND 값이 출력 안됨



5. 원인 분석

가. PCB 제조 공정상의 에칭(Etching) 불량 및 레이아웃 간격(Clearance) 설계 미흡

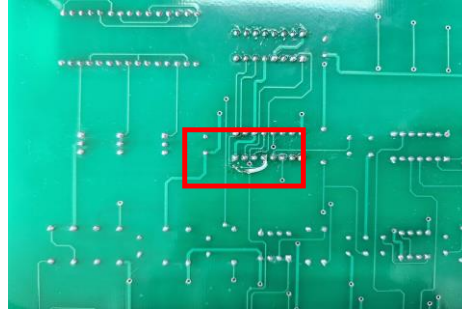
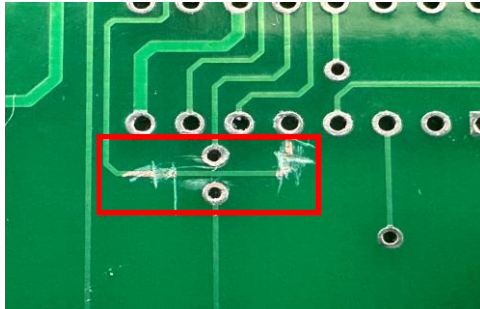


세부 수행 내용

나. 분석 결과 및 검증

→ 제품 수령 후 육안 검사(Visual Inspection) 및 멀티미터 도통 테스트(Continuity Test)를 실시한 결과, 서로 독립적이어야 할 신호 패턴(Trace)과 비아(Via) 사이에서 Short-circuit(단락) 현상을 확인하였습니다.

다. 해결 방안



1) 오결선된 배선 절단 (Cut Trace)

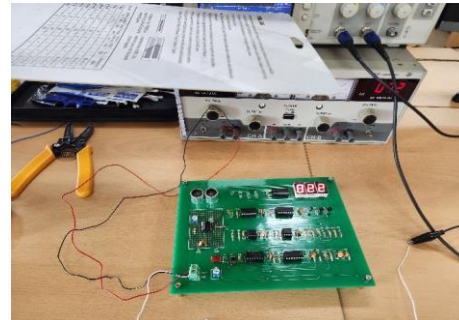
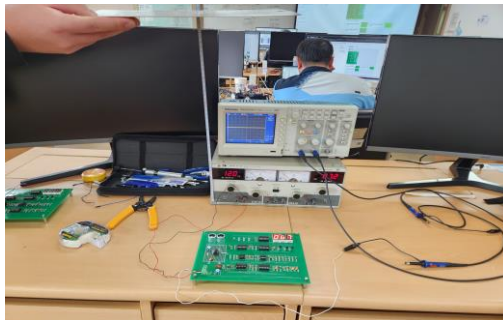
가) 도구를 사용하여 오결선 지점의 구리선을 긁어내어 전기적 연결을 확실하게 분리함

나) 절단 부위 주변의 솔더 레지스트를 제거하여 구리 노출을 최소화함

2) 와이어 연결

가) 배선이 끊어진 상태에서 회로도의 원래 연결도에 맞게 정확한 핀을 확인한 후, 절연 코팅된 와이어를 사용하여 두 점을 직접 납땜으로 연결함.

라. 결과 및 검증 (Verification)



1) 도통 테스트: 수정 작업 완료 후, 멀티미터를 사용하여 끊어낸 배선 부위가 완전히 절단되었는지(무한대 저항)와 와이어가 원래 의도대로 연결되었는지를 확인함.

2) 동작 테스트: 최종적으로 회로에 전원을 공급하고 동작 테스트를 수행한 결과, 초기 오류 현상이 사라지고 회로가 정상적으로 작동함을 확인함.

프로젝트 기술서

작성자

임청수

2	프로젝트명: Smart Grip Car
수행 기간	2026. 01. 23 ~ 2026. 01. 30
담당 역할	1. 초음파 데이터를 이용한 지능형 장애물 감지 2. I2C 방식을 통해 CLCD에 실시간 데이터 디스플레이 3. 조이스틱을 이용한 원격 시스템 구현
수행 목표	1. 통합 제어 : STM32 MCU 기반 HW/SW 통합 제어 프로세스 습득 2. 정밀 주행 : 4륜 메카넘 휠의 독립 제어를 통한 전방향 주행 구현 3. 원격 조종 : 조이스틱 및 UART 통신 기반의 실시간 원격 제어 시스템 구축 4. 지능형 판단 : 초음파/조도 센서 데이터를 활용한 자율 장애물 회피 및 라이트 제어
사용 기술	1. MCU: STM32F103 (ARM Cortex-M3) 2. 언어/툴: C언어, STM32CubeIDE 3. 통신: UART (조이스틱 원격 제어), I2C (LCD 디스플레이) 4. Peripheral: PWM (DC/서보 모터), ADC (조이스틱/조도 센서), Timer Interrupt 5. 액추에이터: L298N 모터 드라이버, 메카넘 휠, 서보 모터(그리퍼)

세부 수행 내용

1. 개요

가. 목적 - "STM32 MCU와 센서 네트워크를 활용하여 전방향 정밀 주행 및 물체 파지가 가능한 지능형 원격 제어 로봇 시스템을 구현한다."

나. 프로젝트 시연 영상



* QR코드를 스캔하면 실제 주행 시연 영상을 확인할 수 있습니다. ≡

다. 소개

- 1) **STM32F103** 기반으로 메카넘 휠 주행과 정밀한 집게(Grip) 기능을 통합한 **지능형** 로봇 플랫폼입니다.
- 2) FreeRTOS를 활용해 센서 데이터 수집과 실시간 원격 제어의 효율성을 극대화한 프로젝트입니다.

2. 개발환경

HARDWARE



SOFTWARE



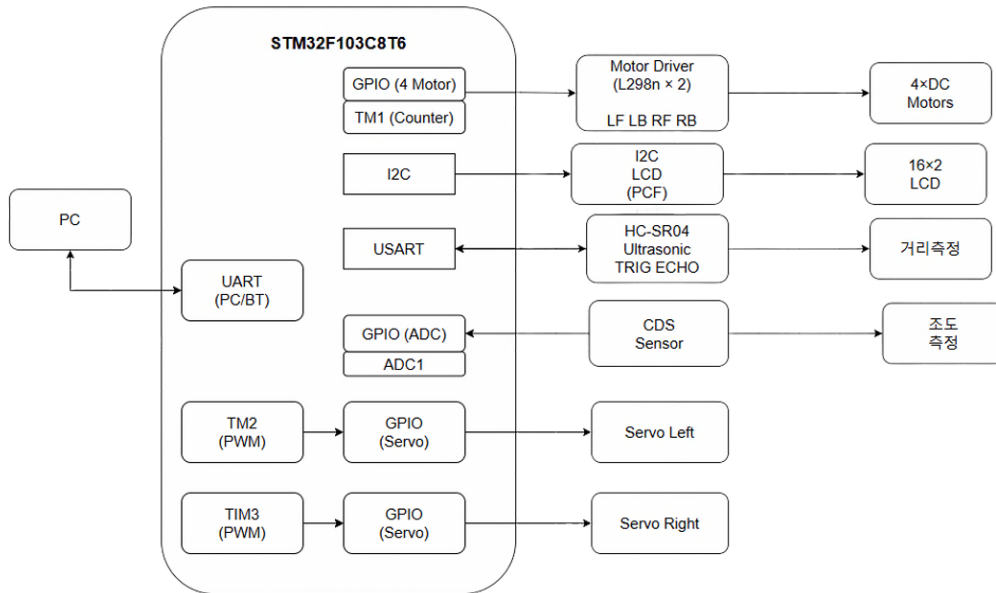
LANGUAGE



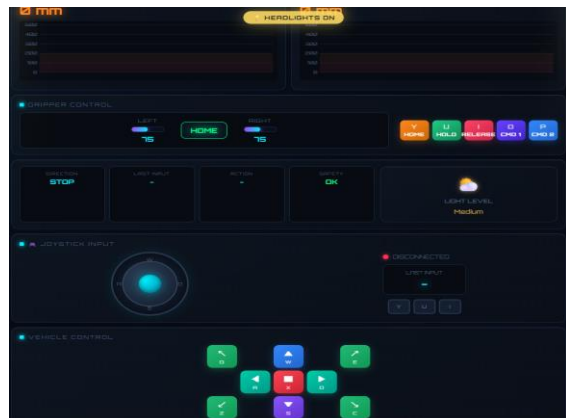
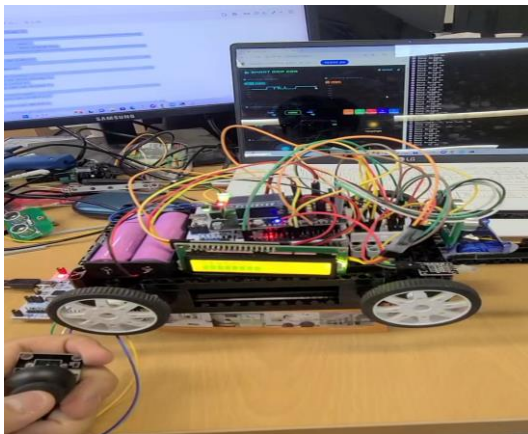
세부 수행 내용

3. 프로젝트 블록도

가. 시스템 블록 다이어그램



4. 주요 기능



- 가. 조이스틱을 활용한 실시간 원격 제어 구현
- 나. 초음파 센서를 활용한 장애물 감지 및 거리 측정 로직 구현
- 다. 2축 서보 모터 기반 그리퍼(집게) 제어
- 라. 조도 센서를 활용한 자동 LED 제어
- 마. 센서 데이터를 LCD를 통한 실시간 데이터 시각화

세부 수행 내용

5. 담당 역할

- 가. 초음파 센서를 이용한 거리 측정 로직 구현
- 나. CLCD 출력 및 거리 데이터 시각화 기능 구현
- 다. UART 통신 기반 원격 제어 기능 구현
- 라. 센서 데이터 처리 및 UART 통신 기반 제어 로직 구현 담당

6. 문제 해결 경험

- 가. 서보 모터 토크 부족 문제 → 물체를 잡는 방식에서 들어 올리는 방식으로 구조 변경
- 나. 조이스틱 버튼 미작동 문제 → 중앙값 재설정 및 코드 수정으로 해결
- 다. 조이스틱과 차량 간 통신 문제 → pull-up 설정 변경 및 통신 코드 수정
- 라. 명령 누락 및 응답 지연 문제 → Polling 방식에서 Interrupt + Ring Buffer 기반 구조로 개선하여 안정적인 통신을 구현

7. 느낀점

- 가. 팀 프로젝트 → 팀 프로젝트를 통해 협업과 역할 분담의 중요성을 배웠으며, 시스템 통합 과정에서 발생한 문제를 해결하며 협업을 통한 문제 해결 능력을 향상 시킬 수 있었습니다.
- 나. 제어 및 센서 문제 해결 → 조이스틱 중앙값 불안정과 초음파 센서 데이터 오차 문제를 겪었지만, 값 보정과 반복 테스트를 통해 안정적으로 동작하도록 개선하였습니다.

프로젝트 기술서

작성자
임청수

3	프로젝트명: Vision AI 기반 스마트 푸드 재고 관리 및 모니터링 시스템
수행 기간	2026. 03. 05 ~ 2026. 03. 11
담당 역할	1. 시스템 아키텍처 설계 2. AI 모델 학습 3. 영상 처리 알고리즘 구현 4. 데이터 로그 시스템 구축
수행 목표	1. YOLOv8 기반의 실시간 다중 객체(접시) 탐지 모델 최적화 (mAP50 0.99 달성) 2. HSV 색공간 분석을 활용한 정밀 잔량(%) 수치화 알고리즘 개발 3. 데이터 신뢰도 향상을 위한 필터링 알고리즘 및 CSV 로깅 시스템 구축
사용 기술	1. 개발 툴 : PyCharm, Google Colab, Roboflow (데이터 라벨링) 2. 언어 및 라이브러리 : Python, OpenCV, YOLOv8, Pandas 3. 적용 기술: Object Detection(YOLO), Digital Image Processing(HSV Masking), Data Smoothing (Moving Average Filter)

세부 수행 내용

0. 사전 미니프로젝트



파이썬을 이용하여 외부 라이브러리와 직접 구현
알고리즘을 모두 지원하는 영상처리 구현



OpenCV 없이 구현한 C와 C++
기반 영상처리 프로그래밍

1. 개요

- 가. 목적 - Vision AI 기반 실시간 잔량 분석을 통한 뷔페 운영 효율화 및 식자재 비용 절감
나. 동작영상 - <https://youtu.be/QgF-QQFg53Q>
다. 프로젝트 시연 영상



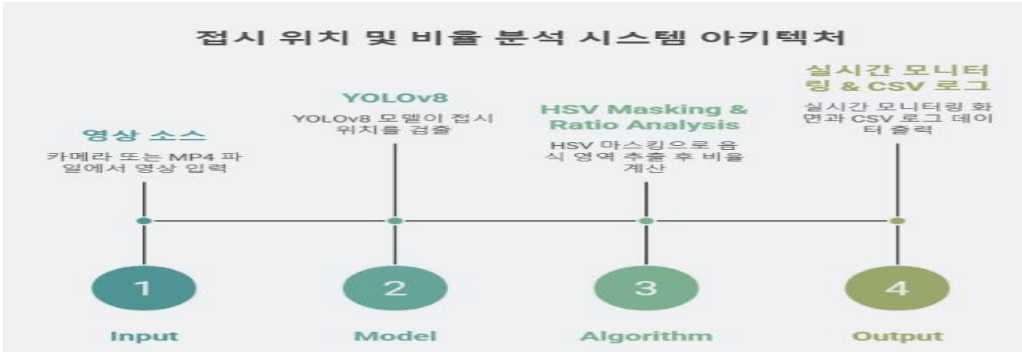
* QR코드를 스캔하면 실제 시연 영상을 확인할 수 있습니다.≡

라. 소개

- 1) 딥러닝(YOLOv8)과 HSV 분석을 결합하여 접시 위의 음식 잔량을 실시간 확인 및 관리
2) 각 접시에 고유 ID를 부여하고, 분석된 데이터를 로그로 저장하여 '데이터 자산화'에 초점

2. 시스템 구성요소

- 가. 시스템 아키텍처



세부 수행 내용

나. 핵심 요약 (Summary)

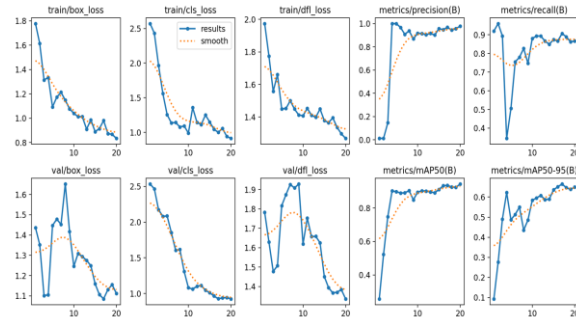
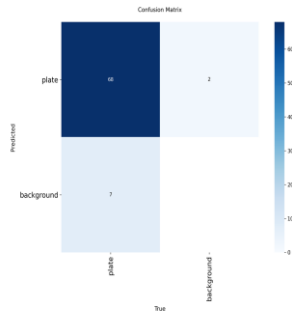
- 1) **하이브리드 분석** : 지도학습(YOLOv8)의 '강건한 탐지'와 비지도/수식기반(HSV Masking)의 '정밀 분석' 결합
- 2) **데이터 신뢰성 확보** : 5-Frame Smoothing 필터를 통해 실시간 배식 중 발생하는 노이즈(식기 가림 등) 차단.
- 3) **운영 데이터 자산화** : 실시간 분석 결과를 단순 시각화하는 데 그치지 않고, ID, 잔량(%), 상태(Status) 값을 CSV 로그로 자동 저장하여 향후 빅데이터 기반의 메뉴 수요 예측 및 배식 최적화가 가능하도록 설계했습니다.

다. 기능 소개 (Main Features)



1) 모델 성능 분석 (Technical Performance)

- **mAP50 (B)** : 약 15 Epoch 이후 0.8(80%) 이상의 높은 정확도 수준에서 수렴.
이는 다양한 조명 환경에서도 접시 객체를 명확히 구분함을 의미함.
- **Precision & Recall** : 정밀도와 재현율이 균형 있게 상승하며, 특히 ****Recall(재현율)****이 높게 유지되어 접시를 놓치는 경우(False Negative)를 최소화함.
- **Loss Analysis** : box_loss와 dfl_loss가 매끄럽게 하강하며 과적합(Overfitting) 없이 안정적으로 학습됨을 확인.



2) 핵심 알고리즘의 시각화 및 논리화

가) Step 1. YOLOv8 Object Detection (지도학습)

- 100여 장의 정밀 라벨링 데이터를 통해 학습된 모델이 접시의 위치(Bounding Box)를 실시간 추적
- **핵심** : 배경 노이즈를 제거하고 분석할 '영역(ROI)'만 정확히 추출.

나) Step 2. HSV Color Space Masking

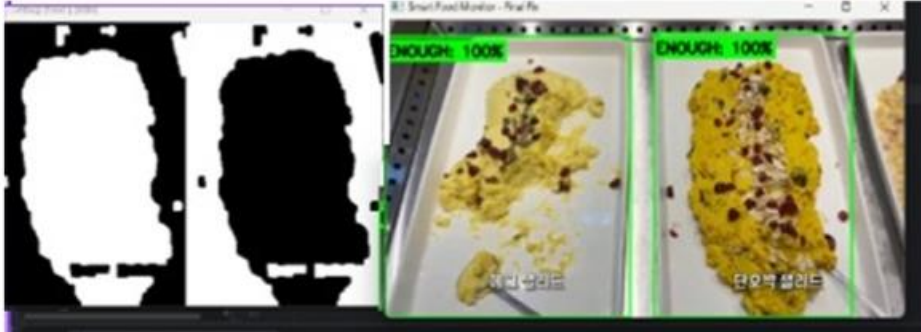
- 추출된 접시 영역 내에서 음식물에 해당하는 색상 범위만 필터링.
- RGB 대비 조명 변화에 강한 HSV 모델을 사용하여 인식 오류 최소화.

다) Step 3. Pixel Count & Ratio Calculation

- 마스크된 픽셀 수를 전체 접시 면적과 대비하여 ****잔량(%)****으로 수치화.
- 수치에 따라 ENOUGH(녹색), PREPARE(황색), FILL(적색) 상태값 부여.

세부 수행 내용

라. 오류 분석 및 개선 (Error Analysis & Improvement)



1) [Issue] 밝은 색상 음식의 인식 저하 및 잔량 오차

가) 문제 현상 : 에그샐러드, 감자샐러드 등 밝은 색상의 음식이 흰색 접시 위에 놓일 경우, HSV 마스킹 과정에서 배경(접시)과 음식을 구분하지 못해 잔량이 0%로 수렴하거나 수치가 급격히 튀는 현상 발생.

나) 원인 분석 :

- 색상 유사성: 음식과 접시의 채도(Saturation) 및 명도(Value) 차이가 미미함.
- 조명 영향: 실내 조명의 반사광으로 인해 특정 영역의 픽셀 값이 화이트아웃(White-out)됨.

2) [Solution] 하이브리드 보정 로직 적용

가) 다중 임계값 설정 (Multi-Thresholding) : 단일 HSV 범위를 사용하는 대신, 밝은 색상 전용 마스크를 추가로 설계하여 미세한 채도 차이를 감지할 수 있도록 알고리즘을 고도화했습니다.

나) 에지 검출 결합 (Canny Edge Detection) : 색상 정보만으로는 구분이 어려운 경우, 음식의 질감(Texture)과 경계선을 파악하는 에지 검출 알고리즘을 병행 적용하여 객체의 영역(ROI)을 더욱 명확히 확정했습니다.

다) 시간축 필터링 (Temporal Filtering)

- 5-Frame Moving Average : 배식 중 일시적으로 발생하는 노이즈나 조명 반사를 차단하기 위해 5프레임 이동 평균 필터를 적용.
- 결과 : 수치 변동 폭이 기존 대비 약 70% 감소하여 안정적인 데이터 확보에 성공.

3) [Result] 개선 전후 비교

가) 개선 전 : 에그샐러드 인식을 약 65% (잔량 오차 ±15%)

나) 개선 후 : 에그샐러드 인식을 약 92% (잔량 오차 ±3% 이내)

다) 시사점 : 환경(조명, 식기 색상) 변화에도 유연하게 대응할 수 있는 영상 처리 파이프라인 구축.

3. 개선사항

가. 기능적 보완

1) 다양한 조명 환경 적응 :

실습실 환경 외에 실제 식당처럼 조명이 어둡거나 노란빛이 도는 곳에서도 음식을 잘 구분할 수 있도록 다양한 데이터셋을 추가

2) 사용자 알림 시스템 연동 :

현재는 화면에 상태가 표시되지만, 실제 현장에서는 관리자가 자리를 비울 수도 있습니다. 이를 대비해 잔량이 부족할 때 스마트폰 앱이나 메신저로 알림을 보내는 기능을 적용 예정

나. 데이터 활용

1) 데이터 활용 :

요일/시간별 소비 패턴 기록 : 단순히 지금의 상태를 보는 것을 넘어, 어떤 메뉴가 언제 빨리 소진되는지 데이터를 모을 예정이며, 이를 통해 "금요일 점심에는 제육볶음을 평소보다 1.5배 준비해야 한다"는 식의 정보를 제공하는 시스템으로 변경 예정

4. 느낀점

가. 데이터의 중요성

알고리즘만큼이나 양질의 학습 데이터(각도, 밝기 등)가 성능에 직결됨을 파악

나. 도구 활용 능력

Roboflow를 통한 라벨링 및 'YOLO' 모델 학습 프로세스 전반에 대한 이해

프로젝트 기술서

작성자
임청수

4	프로젝트명: Smart Grip Car Ver1.1
수행 기간	2026. 03. 25 ~ 2026. 04. 01
담당 역할	1. AI 모델 학습 2. 시스템 아키텍처 설계 3. 펌웨어 연동 4. 전체 시스템 통합
수행 목표	1. AI와 임베디드 융합: 라즈베리 파이 4 기반 YOLOv8 실시간 추론 + STM32 연동 2. 비전 기반 자동화: AI 카메라를 통한 환경 인식 및 자율 판단 로직 구현 3. 풀스택 시스템: Python / Node.js / Flask / C 다중 언어 통합 로봇 제어 4. 웹 UI 실시간 제어: 저지연 스트리밍 및 브라우저 기반 차량 조작
사용 기술	1. H/W: Raspberry Pi 4, NUCLEO-F103RB (STM32F103), L298N 모터 드라이버, HC-SR04 초음파, CDS 조도 센서, 서보 모터 2. S/W: Python (Flask, YOLOv8), Node.js (Express), C (STM32 펌웨어), OpenCV, PyTorch 3. 통신: UART (Serial), I2C (LCD), PWM (모터/서보), ADC (센서) 4. AI 학습: Google Colab (NVIDIA T4), TACO 데이터셋, YOLOv8n

세부 수행 내용

1. 개요

- 가. 목적
- 1) AI와 임베디드의 융합: 라즈베리 파이 4 기반 영상처리와 임베디드 소프트웨어를 이용하여 STM32와 PC 시뮬레이션 기반 시스템을 하나의 통합 임베디드 시스템으로 구축한다
- 2) 비전 기반 자동화: 단순 원격 조종을 넘어 AI 카메라를 통한 환경 인식 및 판단 로직 습득
- 3) 시스템 통합 설계: Python(AI), Node.js(Server), Flask, C(Firmware) 등 다양한 언어를 활용한 풀스택 로봇 제어 구현

나. 프로젝트 시연 영상



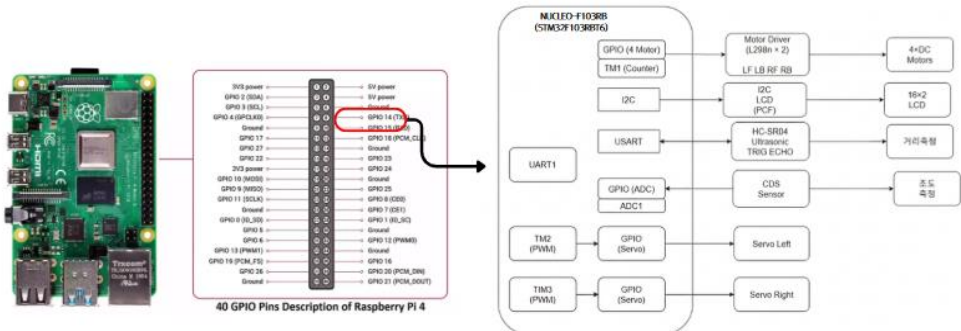
* QR코드를 스캔하면 실제 수행 시연 영상을 확인할 수 있습니다.

다. 소개

- 1) 라즈베리 파이 4와 STM32를 이중 컨트롤러로 연결하고, YOLOv8 AI 카메라로 쓰레기-용기류를 실시간 인식해 포크리프트 방식으로 물체를 집어 올리는 자율 이송 로봇 플랫폼입니다.
- 2) Python(Flask), Node.js(Express), C(펌웨어)를 하나로 통합하여 웹 브라우저에서 실시간 영상을 확인하며 차량을 직접 제어할 수 있도록 구현한 풀스택 임베디드 시스템입니다.

2. 시스템 구성요소

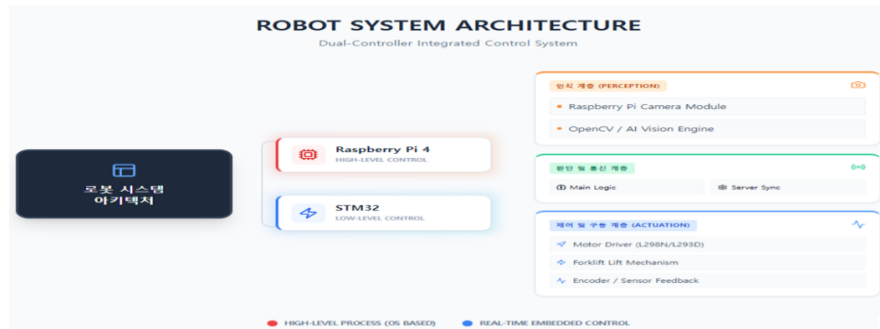
가. 프로젝트 블럭도



세부 수행 내용

STM32 주변장치	연결 방식	기능
GPIO (4 Motor)	L298N × 2 → LF LB RF RB	4륜 DC 모터 독립 구동
TM1 (Counter)	엔코더 피드백	주행 속도 및 거리 측정
I2C	PCF → 16×2 LCD	실시간 센서 데이터 표시
USART (UART1)	HC-SR04 (TRIG/ECHO)	초음파 거리 측정
ADC1	CDS Sensor	주변 조도 측정
TM2 (PWM)	GPIO → Servo Left	리프트 좌측 서보 제어
TM3 (PWM)	GPIO → Servo Right	리프트 우측 서보 제어

나. 시스템 아키텍처



1) Data Flow

- 가) 카메라 영상이 Raspberry Pi 4의 YOLOv8 기반 비전 처리부로 전달되어 객체를 인식
- 나) Flask 서버가 실시간 카메라 스트리밍을 담당
- 다) Node.js 서버가 웹 UI 제어 명령을 처리
- 라) Node.js 서버가 UART 통신을 통해 STM32로 주행 및 그리퍼 제어 명령
- 마) STM32는 수신된 명령에 따라 DC 모터, 서보 모터, 센서 제어 수행
- 바) 센서 및 상태 데이터는 웹 UI에서 실시간 모니터링

3. 개발환경 및 도구

가. AI 및 데이터 학습 환경

실시간 사물 인식을 위한 딥러닝 모델 학습 및 추론 환경입니다.

구분	항목	상세 내용
AI 모델	YOLOv8n	Ultralytics Nano (경량화 모델)
학습 장치	Google Colab	NVIDIA T4 GPU 가속 환경
데이터셋	TACO Data set	쓰레기/용기 분류 특화 데이터셋 (Customizing)
추론 엔진	PyTorch / OpenCV	실시간 객체 탐지 및 이미지 전처리

나. 통합 제어 및 서버 아키텍처

구분	항목	상세 내용
OS / 플랫폼	Raspberry Pi OS / Node.js v24.13.0	라즈베리 파이 기반 통합 제어 플랫폼
메인 컨트롤러	Express (Node.js)	웹 인터페이스 제공 및 시스템 로직 관리
비전 서버	Flask (Python)	YOLOv8 기반 실시간 영상 스트리밍 서버
메인 컨트롤러	Express (Node.js)	웹 인터페이스 제공 및 시스템 로직 관리
통신 방식	UART (Serial)	Node.js(서버) ↔ STM32(하드웨어) 간 제어 명령 및 데이터 송수신

세부 수행 내용

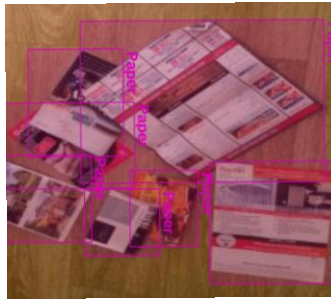
4. 주요 업데이트 및 성능 개선 (v1.0 → v1.1)

항목	v1.0 (기존)	v1.1 (현재)
메인 보드	STM32 단독	Raspberry Pi 4 + STM32
제어 방식	조이스틱 / UART	웹 브라우저 UI / 비전 AI 자동 제어
인식 센서	초음파, 조도	YOLOv8 AI 카메라, 초음파
그리퍼 구조	좌우 집게 방식(Gripper)	상하 리프트 방식(Forklift)
학습 데이터	없음	TACO 데이터셋 (Bottle, Cup 등, 50 Epochs)

5. 데이터셋 및 학습 (Data set)

가. TACO (Trash Annotations in Context) 데이터셋

실외·실내 환경에서 촬영한 쓰레기/용기 분류 특화 데이터셋을 사용했습니다.

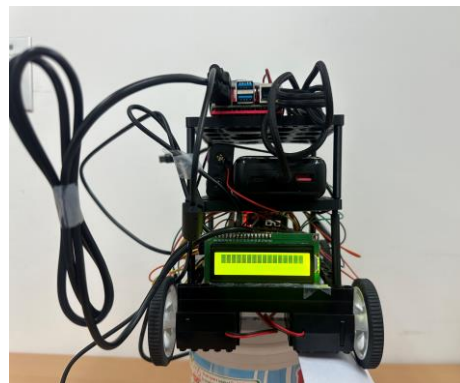
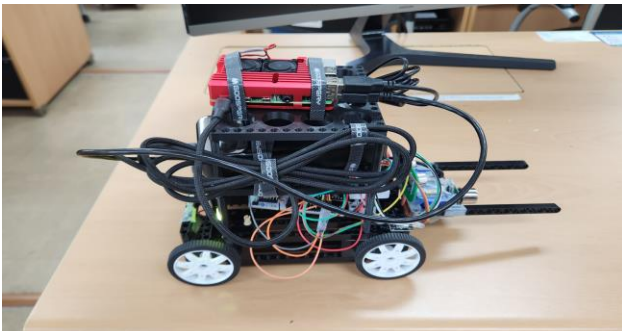


나. 학습 모델 및 환경 (Training Environment)

항목	내용
사용 모델	YOLOv8n (Ultralytics Nano) - 라즈베리 파이 경량화 모델
학습 환경	Google Colab (Tesla T4 GPU 활용)
데이터셋	TACO Dataset + 커스텀 데이터
전체 이미지	원본 약 2,500장, 증강 후 총 6,004장
분할	Train 4,200장 / Valid 1,704장 / Test 100장
Epochs	50
Classes	18개 세부 클래스

6. 테스트 및 결과

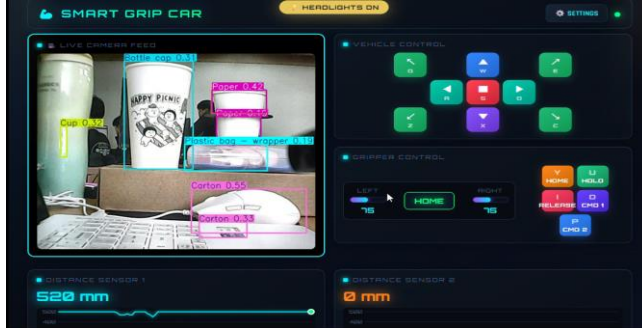
가. 차량 완성 사진



세부 수행 내용

나. 실시간 데이터 동기화 및 모니터링

웹 UI에서 YOLOv8 객체 인식 화면, 거리 센서 데이터, 그리퍼 제어 상태를 실시간으로 확인할 수 있도록 구성했습니다.



```

[1] Carton conf=0.24
[12] Plastic bag - wrapper conf=0.24
[5] Carton conf=0.42
[1] Bottle cap conf=0.17
[5] Carton conf=0.16
[5] Carton conf=0.23
[1] Bottle cap conf=0.23
[12] Plastic bag - wrapper conf=0.21
[5] Carton conf=0.19
[1] Bottle cap conf=0.39
[5] Carton conf=0.38
[5] Carton conf=0.30
[1] Bottle cap conf=0.28
[1] Bottle cap conf=0.41
[6] Cigarette conf=0.30
[5] Carton conf=0.22
[1] Bottle cap conf=0.51
[5] Carton conf=0.34
[1] Bottle cap conf=0.19
[5] Carton conf=0.71
[5] Carton conf=0.94
[12] Plastic bag - wrapper conf=0.15
[5] Carton conf=0.92
[12] Plastic bag - wrapper conf=0.50
[5] Carton conf=0.16
[5] Carton conf=0.92
[12] Plastic bag - wrapper conf=0.19
  
```

다. 초기 주행 테스트 문제 원인 분석

- 환경적 요인 (Environmental Factors):
 - 실내 조도 변화와 역광으로 인해 객체 경계가 흐려져 인식이 저하됨
 - 복잡한 배경과 바닥 패턴을 객체로 오인식하는 현상 발생
- 데이터셋의 한계 (Dataset Limitations):
 - 로봇의 낮은 카메라 시점(Low-view)에 맞는 학습 데이터 부족
 - 파손된 캔, 훼손된 라벨, 비정형 객체에 대한 데이터 부족
- 하드웨어 제약 (Hardware Constraint):
 - 주행 중 카메라 진동으로 Motion Blur 발생
 - Raspberry Pi 추론 속도 확보를 위한 해상도 조정 과정에서 작은 객체 특징 손실

7. 시행착오 및 해결방안

시행착오	해결방안
TACO 데이터셋과 실제 국내 실내 환경 차이로 인식을 저하	국내 제품 및 로봇 Low-view 시점 이미지를 직접 수집하여 데이터 보강
조도 변화, 배경 간섭으로 클래스 혼동 발생	커스텀 데이터 추가 및 데이터 증강으로 환경 적응력 개선
로컬 환경의 GPU 성능 부족으로 학습 속도 저하	Google Colab Tesla T4 GPU를 활용하여 학습 시간 단축
Raspberry Pi 실시간 추론 시 지연 발생	YOLOv8n 경량 모델 적용 및 해상도 조정으로 처리 속도 개선

8. 느낀점

가. AI와 임베디드 통합 경험:

YOLOv8 기반 영상 인식, STM32 펌웨어 개발, Node.js 서버 구축까지 전 과정을 수행하며 AI, 임베디드, 서버 기술이 통합되는 시스템 구조를 이해할 수 있었습니다.

나. 하드웨어 제약의 중요성:

라즈베리 파이의 제한된 연산 자원을 고려하며 성능과 시스템 안정성 간의 균형을 설계하는 경험을 쌓았습니다.

다. 데이터 품질의 중요성:

실제 테스트 과정에서 학습 데이터와 운영 환경의 차이에 따라 인식 성능이 달라지는 것을 확인하였습니다. 이를 통해 데이터 수집 및 전처리 과정이 모델 성능에 직접적인 영향을 미친다는 점을 이해하였습니다.

라. 시스템 통합 문제 해결:

다양한 하드웨어와 소프트웨어를 연동하는 과정에서 발생한 문제를 해결하며 체계적인 디버깅 능력과 개발 과정의 기록 관리 중요성을 체득하였습니다.

프로젝트 기술서

작성자

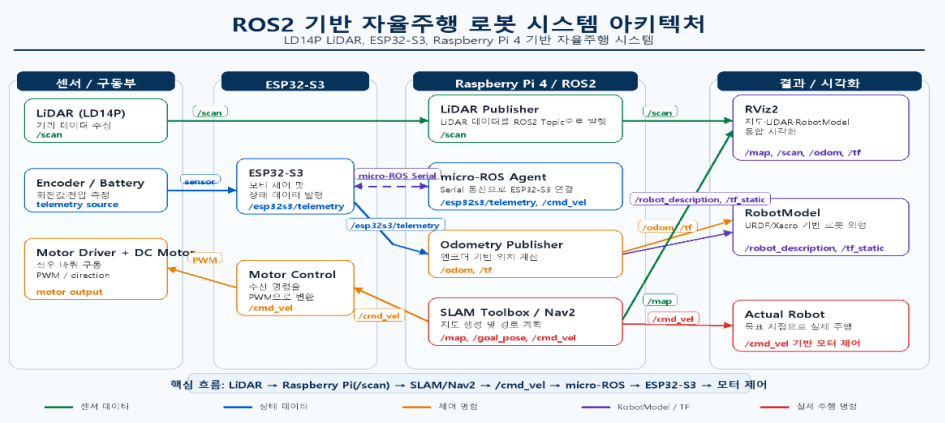
임청수

5	프로젝트명: ROS2 기반 코봇 자율주행 시스템 개발
수행 기간	2026. 05. 11 ~ 2026. 05. 29
담당 역할	1. ROS2 Humble 기반 자율주행 시스템 구축 2. ESP32-S3 ↔ Raspberry Pi4 micro-ROS 통신 구현 3. LiDAR 기반 SLAM 지도 생성 기능 구현 4. Nav2 기반 목표 지점 자율주행 구현 5. 회로도 및 PCB 설계, 3D 모델링을 통한 하드웨어 확장 구조 설계
수행 목표	1. ROS2 기반 분산형 로봇 제어 시스템 구축 2. LiDAR를 활용한 실내 환경 지도 생성(SLAM) 3. Nav2를 활용한 목표 지점 자율주행 구현 4. micro-ROS 기반 MCU-ROS2 연동 5. RViz2 기반 센서 및 로봇 상태 시각화 6. PCB 및 기구 설계를 통한 실제 로봇 제작
사용 기술	SBC: Raspberry Pi 4 / MCU: ESP32-S3 / OS: Ubuntu 22.04 / Framework: ROS2 Humble, micro ROS / Sensor: LD14P LiDAR / 통신: UART Serial / Mapping: SLAM Toolbox / Navigation: Nav2 / Visualization: RViz2 / 회로/PCB 설계: EasyEDA / 기구 설계: SolidWorks / 제작: Bambu 3D 프린터 / 언어: C++, Python
세부 수행 내용	
1. 개요 가. 목적 1) LiDAR 기반 실내 환경 지도 생성(SLAM) 2) Nav2 기반 목표 지점 자율주행 구현 3) micro-ROS 기반 ESP32-S3와 Raspberry Pi4 간 분산 제어 시스템 구축 4) RViz2 기반 지도·센서·로봇 상태 통합 시각화 나. 프로젝트 시연 영상  * QR코드를 스캔하면 실제 주행 시연 영상을 확인할 수 있습니다. 다. 소개 1) Raspberry Pi4와 ESP32-S3를 micro-ROS로 연동하여 ROS2 기반 자율주행 로봇 시스템을 구현하였습니다. 2) LD14P LiDAR를 활용하여 실시간 환경 데이터를 수집하고, SLAM Toolbox를 이용해 실내 지도 생성 기능을 구현하였습니다. 3) Nav2를 활용하여 목표 지점까지의 경로를 계획하고, RViz2를 통해 지도·센서·로봇 상태를 통합 시각화하는 자율주행 시스템을 구축하였습니다. 4) Raspberry Pi 4, ESP32-S3, LD14P LiDAR, 모터 드라이버, DC 모터·엔코더를 연결하기 위한 로봇 제어 PCB를 EasyEDA로 설계하였고, SolidWorks와 Bambu 3D 프린터를 활용하여 하드웨어 보강 구조를 제작하였습니다.	

세부 수행 내용

2. ROS2 시스템 아키텍처

가. 프로젝트 블록도



* 핵심 흐름: LiDAR → Raspberry Pi(/scan) → SLAM/Nav2 → /cmd_vel → micro-ROS → ESP32-S3 → 모터 제어
LiDAR 데이터는 ROS2 Topic으로 전달되며, Nav2의 /cmd_vel 명령은 ESP32-S3를 통해 실제 모터 제어에 연결된다.

3. 핵심 기능 구현

가. micro-ROS 통신 (ESP32-S3 ↔ Raspberry Pi4 UART Serial 통신)

- Raspberry Pi 4에서 micro_ros_agent를 실행하여 UART Serial로 ESP32-S3와 연결하고, /cmd_vel 구독/
esp32s3/telemetry 발행 구조로 ROS2 고수준 제어와 MCU 저수준 제어를 분리하였습니다.

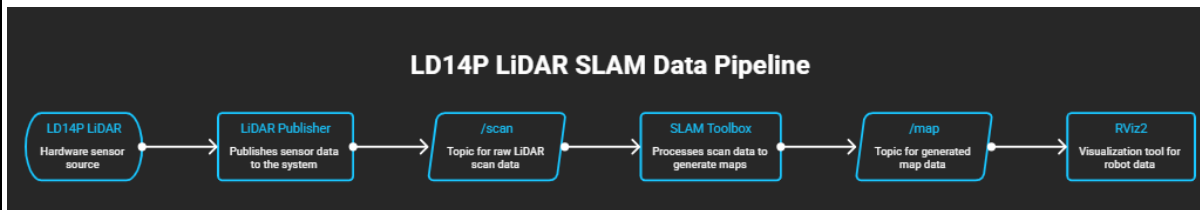
나. 주요 Topic 데이터 흐름

```
root@cs:~/ros2_ws# ros2 topic list | grep -E "scan|odom|cmd_vel|map|tf"
/cmd_vel
/cmd_vel_nav
/cmd_vel_teleop
/global_costmap/costmap
/global_costmap/costmap_raw
/global_costmap/costmap_updates
/global_costmap/footprint
/global_costmap/global_costmap/transition_event
/global_costmap/published_footprint
/local_costmap/clearing_endpoints
/local_costmap/costmap
/local_costmap/costmap_raw
/local_costmap/costmap_updates
/local_costmap/footprint
/local_costmap/local_costmap/transition_event
/local_costmap/published_footprint
/local_costmap/voxel_grid
/map
/map_server/transition_event
/map_updates
/odom
/scan
/tf
/tf_static
root@cs:~/ros2_ws#
```

Topic	역할
/cmd_vel	Nav2 또는 teleop에서 생성한 로봇 주행 명령
/esp32s3/telemetry	ESP32-S3에서 수신한 엔코더, 배터리 상태 데이터
/scan	LiDAR 센서에서 발행하는 거리 데이터
/odom	엔코더-센서 데이터 기반 로봇 위치 정보
/map	SLAM 또는 map_server에서 제공하는 실내 지도
/tf, /tf_static	map, odom, base_link, laser_frame 좌표 관계

* ros2 topic list 실행 결과 /cmd_vel, /scan, /odom, /map, /tf 등 주요 Topic이 정상 등록된 것을 확인하였으며, Nav2의 costmap 관련 Topic까지 함께 발행되어 경로 계획에 필요한 데이터 흐름이 정상 구성됨을 확인하였다.

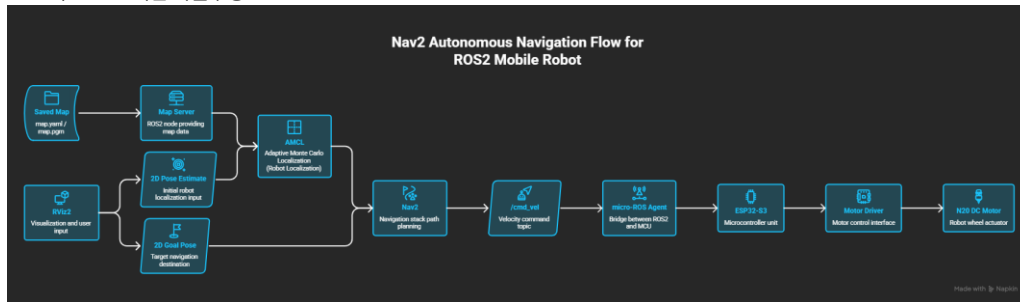
다. LiDAR 기반 SLAM



- LD14P LiDAR를 Raspberry Pi 4에 연결하여 실시간 거리 데이터를 수집하고, LiDAR Publisher로 /scan Topic을 발행하였습니다.
- SLAM Toolbox를 활용해 LiDAR 데이터를 기반으로 실내 환경 지도를 생성하고 로봇 위치를 추정하였습니다.≡
- 생성된 지도(/map)를 RViz2에서 시각화하여 지도 생성 과정을 실시간으로 확인하였습니다.
- 실내 환경을 주행하며 지도 생성 성능을 검증하고, LiDAR 데이터와 실제 환경의 일치 여부를 확인하였습니다.

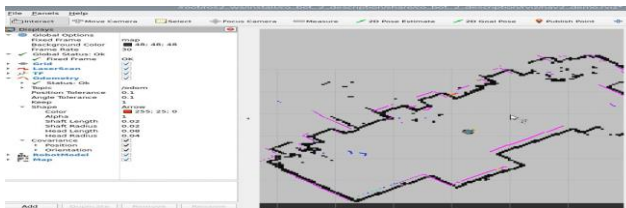
세부 수행 내용

라. Nav2 기반 자율주행



- 1) SLAM Toolbox로 생성한 map.yaml, map.pgm 지도를 Nav2 환경에 적용하였습니다.
- 2) AMCL을 활용하여 로봇의 현재 위치를 지도 상에서 추정하도록 구성하였습니다.
- 3) RViz2의 2D Pose Estimate를 이용하여 초기 위치를 설정하고, 2D Goal Pose를 통해 목표 지점을 지정하였습니다.
- 4) Nav2가 목표 지점까지의 경로를 계획하고 /cmd_vel Topic을 생성하도록 구현하였습니다.
- 5) 생성된 /cmd_vel 명령을 micro-ROS Agent를 통해 ESP32-S3로 전달하여 모터를 제어하였습니다.

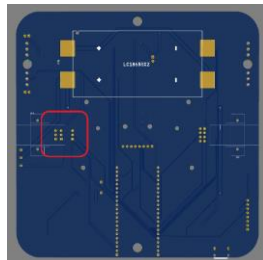
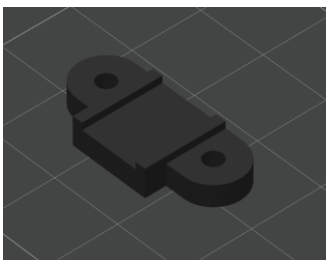
4. RViz2 시각화 결과



```
root@cs:~# ros2 lifecycle get /bt_navigator
active [3]
root@cs:~# ros2 topic info /goal_pose
Type: geometry_msgs/msg/PoseStamped
Publisher count: 1
Subscription count: 1
root@cs:~# ros2 topic echo /cmd_vel
linear:
  x: 0.02526315789473684
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: -0.16
linear:
  x: 0.02526315789473684
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: -0.32
linear:
  x: 0.02526315789473684
  y: 0.0
  z: 0.0
```

* RViz2에서 지도, LiDAR, Odometry, RobotModel 상태를 한 화면에서 확인했고, /goal_pose 수신과 /cmd_vel 명령 생성이 정상적으로 수행되는 것을 확인했습니다.

5. 하드웨어 및 회로 설계



엔코더 핀 간섭 확인 및 모터 보강 브라켓 설계

• Raspberry Pi 4, ESP32-S3, LD14P LiDAR, DC 모터.엔코더, 모터 드라이버, 전원 회로를 기반으로 자율주행 로봇 하드웨어를 구성하였습니다.

• 각 부품의 전원 및 신호 연결을 정리하기 위해 EasyEDA를 활용하여 회로도 및 로봇 제어용 PCB를 설계하였습니다.

• PCB 설계 후 엔코더 핀과 PCB 하단 핀이 간섭되는 문제를 발견하였고, SolidWorks로 모터 보강 브라켓을 설계한 뒤 Bambu 3D 프린터로 출력하여 해결하였습니다.

• 전면 카메라, SPI LCD, 그리퍼를 포함한 확장형 구조를 적용하여 실제 로봇 하드웨어를 제작하였습니다.

세부 수행 내용

6. 트러블슈팅 및 개선

Serial 통신 문제

GPIO14가 gpio-fan 핀과 충돌
→ gpio-fan 핀 변경 및 /dev/serial0 사용으로 해결

micro-ROS Topic 문제

ROS_DOMAIN_ID 불일치로 Topic 미생성
→ ROS_DOMAIN_ID를 0으로 통일하여 해결

Nav2 실행 오류

일부 BT Plugin 로드 실패
→ navigation.yaml에서 미사용 plugin 주석 처리로 해결

RobotModel / RViz2 문제

robot_description 및 RViz2 설정 문제
→ rsp.launch.py 수정, nav2_demo.rviz 저장으로 해결

7. 한계점 및 향후 계획

주행·지도 정확도

좁은 공간 회전 시 경로 추종 불안정,
SLAM 지도 노이즈
→ Nav2 파라미터 튜닝 및 LiDAR 위치
보정 필요

기능 확장

그리퍼·카메라·LCD는 기본 구조만 반영
→ ROS2 Topic 연동으로 조작·인식·상태
표시 기능 확장 예정

서비스 로봇화

현재 자율주행 흐름 구현 완료
→ 자율주행 + 물체 조작 + 상태 표시 등
합 구조로 발전 예정

8. 프로젝트 회고

기술적 성장	ROS2 Topic 기반 분산 제어 구조를 이해하고, micro-ROS로 MCU와 ROS2를 연동하는 방법을 학습했습니다.
문제 해결 경험	Serial 통신, ROS_DOMAIN_ID, Nav2 Plugin, RViz2 설정 문제를 직접 분석하고 해결하며 디버깅 역량을 높였습니다.
프로젝트 성과	LD14P LiDAR 기반 실내 지도 생성, Nav2 목표 지점 자율주행, ESP32-S3와 Raspberry Pi 4를 연동한 ROS2 분산 제어 시스템을 구축했습니다.